

W5_HW1_PracticalNLP_202200274_고민서

ELLT학과 202200274 고민서

1. 과제 목표

이번 과제는 수업에서 다룬 MLP 기반 감성 분류 코드를 출발점으로 삼아, 하이퍼파라미터를 다섯 개 이상 바꾸어 가며 다섯 개의 실험을 진행하고 그중 적어도 하나가 기존 모델의 성능(67.38%)을 넘기도록 만드는 것이 핵심이다. 사용한 데이터셋은 Hugging Face의 [Sp1786/multiclass-sentiment-analysis-dataset](#)이며 negative / neutral / positive 세 클래스로 구성된 다중 분류 문제다.

제약 조건으로는 모델 구조 자체(MLP)를 바꾸는 것이 허용되지 않으며, BERT나 RNN처럼 구조 자체가 다른 모델을 사용하는 것도 안 된다. 따라서 성능을 끌어올리기 위해 손댈 수 있는 부분은 입력 표현(벡터화 방법)과 학습 관련 하이퍼파라미터, 그리고 일부 전처리 정도로 정리된다.

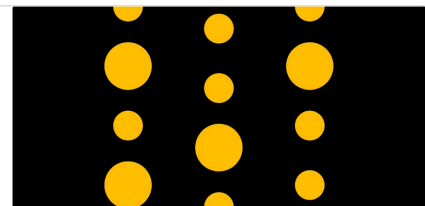
2. 실험 설계 방법

벡터화 방법이 모델 성능에 미치는 영향이 클 것이라고 판단했기 때문에, 실험을 크게 두 축으로 나누어 설계했다. 한 축은 베이스라인과 같은 BoW 표현을 그대로 두고 학습 관련 하이퍼파라미터만 자동 탐색으로 최적화해보는 방향이고, 다른 한 축은 벡터화 방법을 TF-IDF, GloVe, MiniLM, MPNet으로 바꿔가며 어떤 표현이 이 데이터셋에 가장 잘 맞는지를 비교하는 방향이다. 모든 실험의 경우 Google Colab 에서 진행했으며, 런타임 유형 Python3, T4 GPU와 같은 동일한 환경에서 실험을 진행하였다.

하이퍼파라미터 탐색의 경우 일일이 최적의 파라미터를 찾기에는 조합 수가 너무 많기 때문에 Weights & Biases의 Bayesian Sweep을 사용했다. 한 번 sweep을 돌릴 때마다 약 20회 실험을 돌려, 가장 좋은 dev 성능을 기록한 설정을 골라 최종 평가에 사용했다.

모든 실험의 WandB 추적 결과는 아래의 사이트에서 상세히 확인할 수 있다.

imeanseor_
Weights & Biases, developer tools for machine learning
 https://wandb.ai/imeanseor_/nlp-hw1?nw=nwuseraileen02ko



전반적인 모든 실험 과정과 결과는 아래의 깃허브에 상세하게 기록해두었다.

<https://github.com/imeanseor/deeplearning-hyperparameter-experiment>

3. 변경한 하이퍼파라미터의 의미와 선택 이유

과제에서 요구한 다섯 개 이상이라는 조건을 충분히 넘기도록, 아래 여덟 가지 하이퍼파라미터를 실험에 따라 함께 또는 따로 조정했다. 각 항목이 무엇을 의미하고 왜 이번 과제에서 변경 대상으로 삼았는지를 먼저 정리하고, 구체적으로 어떤 값이 선택되었는지는 아래 실험별 분석에서 다시 자세히 다룰 예정이다.

1. **학습률(`learning_rate`)** — 매 학습 스텝에서 가중치를 얼마나 크게 갱신할지를 결정하는 값이다. 너무 크면 손실이 발산하고 너무 작으면 학습이 느려진다. 베이스라인의 $1e-4$ 가 이 데이터에 굳이 최적이라고 확신할 수 없었고, 특히 벡터화 방식에 따라 적절한 스케일이 크게 달라질 가능성이 높아(사전학습 임베딩 위에서는 더 작아야 하는 경우가 많다) 가장 먼저 손을 대야 하는 항목이라고 판단했다.
2. **은닉층 크기(`hidden_size`)** — MLP의 중간 표현력을 결정하는 값이다. 너무 작으면 과소적합, 너무 크면 과적합 위험이 커진다. 베이스라인은 1000으로 적지 않은 용량이지만 입력이 단순한 키워드 신호 중심이라면 그만한 표현력이 필요 없을 수 있다고 판단해 256에서 2000 사이를 같이 탐색했다.
3. **드롭아웃 비율(`dropout`)** — 학습 중 일부 뉴런을 무작위로 비활성화해 과적합을 줄이는 정규화 기법이다. 베이스라인에는 명시적인 정규화가 거의 없었고, dev와 test 사이의 격차를 줄이는 데 가장 직접적인 수단이라고 보아 0.0에서 0.5 사이를 실험했다.
4. **weight decay 계수** — L2 정규화의 강도로, 가중치가 과도하게 커지는 것을 제약해 일반화를 돕는다. 드롭아웃과 함께 과적합 제어의 두 축을 이루는 항목이기 때문에, 둘의 조합이 어떤 지점에서 가장 좋은 결과를 내는지 보고 싶어 0에서 $1e-3$ 범위를 같이 움직였다.
5. **배치 크기(`batch_size`)** — 한 번의 그래디언트 추정에 사용하는 표본 수다. 작을수록 노이즈가 많아 정규화 효과가 있고 클수록 학습이 안정적이지만 일반화에 불리한 경우가 있다. 베이스라인의 128이 최적이라는 보장이 없어 64·128·256 세 값을 같이 넣어 학습률·에폭과의 상호작용까지 함께 살펴보고자 했다.
6. **학습 에폭 수(`num_epochs`)** — 데이터셋을 몇 번 반복해 학습할지를 결정한다. 베이스라인의 10 에폭은 모델이 충분히 학습하기에 짧은 과소적합이 낳을 가능성이 있다고 보아 20·30·50 에폭까지 확장했고, 드롭아웃·weight decay와 결합될 때 과적합 지점이 어디서 나타나는지도 함께 확인했다.
7. **벡터화 방법(BoW / TF-IDF / GloVe300 / MiniLM / MPNet)** — 입력 문장을 숫자 표현으로 바꾸는 방식 자체이다. 일반적인 하이퍼파라미터라기보다 "이 데이터에 어떤 표현이 맞는가"라는 가설을 바꾸는 선택이기 때문에, 이번 과제에서 가장 큰 성능 차이를 만들 수 있는 축이라고 판단했다. 인기 모델을 단순히 모아 본 것이 아니라, BoW를 출발점으로 표현 패러다임을 한 단계씩 다르게 바꾸어가며 비교할 수 있도록 다음 네 가지를 골랐다.
 - **TF-IDF(`sklearn.feature_extraction.text.TfidfVectorizer`)** — BoW의 가장 자연스러운 확장으로, "단어가 등장했는가"에 더해 "이 단어가 얼마나 특이하게 중요한가"까지 가중치로 반영한다. BoW와 동일한 sparse 벡터 형태라 모델 코드 변경이 거의 없어, 단어 가중치 부여만으로 성능을 어느 정도 끌어올릴 수 있는지를 깔끔하게 검증할 수 있다고 보아 첫 번째 표현 변경 후보로 골랐다.
 - **GloVe 300d(`gensim.downloader` 의 `glove-wiki-gigaword-300`)** — 분산 표현의 대표 격이며 word2vec과 함께 가장 널리 쓰이는 사전학습 단어 임베딩이다. Wikipedia와 Gigaword를 합쳐 약 60억 토큰으로 학습되어 일반 영어 어휘 커버리지가 넓고 `gensim` 을 통해 한 줄로 받아 쓸 수 있다. 100d·200d·300d 중 표현력이 가장 풍부한 300d를 골라, 단어 단위 의미 벡터가 sparse 표현보다 유리한지 직접 확인하고자 했다.
 - **MiniLM 384d(`sentence-transformers/all-MiniLM-L6-v2`)** — sentence-transformers 라이브러리에서 사실상 기본값처럼 쓰이는 문장 임베딩 모델이다. 6 레이어 구조라 다운로드와 추론이 모두 빠르고 384차원으로 가벼워 sweep을 20회씩 반복해야 하는 이번 실험 환경과 잘 맞아, 단어 평균이 아니라 문장 전체의 문맥을 담는 표현이 이 데이터셋에 도움이 되는지 확인할 첫 트랜스포머 후보로 선택했다.
 - **MPNet 768d(`sentence-transformers/all-mpnet-base-v2`)** — 같은 sentence-transformers 라이브러리에서 더 크고 강한 표현으로 자주 비교되는 모델이다. MiniLM과 인터페이스가 동일해서 모델 이름만 바꾸면 같은 코드로 돌릴 수 있고, 차원이 384에서 768로 약 두 배 커졌을 때 성능이 정말 따라 오르는지를 직접 검증하기 위해 골랐다.

이렇게 "같은 sklearn 안에서 단순 빈도 → 중요도 가중치", "같은 sentence-transformers 안에서 작은 MiniLM → 큰 MPNet"으로 단계를 두어 배치한 이유는, 라이브러리 변경에 따른 노이즈를 최소화하면서

"표현이 정교해질수록 이 데이터셋에서 정말 더 좋은 결과가 나오는가"라는 가설을 깔끔하게 검증하기 위해서였다.

8. **TF-IDF 세부 옵션**(`max_features`, `min_df`, `n-gram`, 전처리 함수 등) — 가장 좋은 성능을 낸 TF-IDF 표현을 한 번 더 끌어올리기 위해 조정한 항목들이다. `max_features` 는 사용할 어휘 수를 제한해 노이즈 단어를 줄이고 `min_df` 는 너무 드물게 등장하는 단어를 제외해 과적합을 줄이며, `n-gram` 확장은 "not good"과 같은 부정 표현이나 관용구를 단일 단어로 끝내지 않고 두 단어 단위로 잡아내기 위해 도입했다.
같은 TF-IDF 안에서도 이런 세부 설정이 성능에 의미 있게 영향을 주었고 최종 제출 모델인 Exp3-v8의 핵심 개선 원동력이 되었다.

4. 다섯 개 실험의 최종 성능

- **Exp1 (BoW Baseline):** Test 67.38% → 비교 기준
- **Exp2 (BoW + Sweep):** Test 68.26% → 베이스라인 대비 +0.88%p
- **Exp3-v8 (TF-IDF + Sweep + 추가 개선):** Dev 70.05% | Test **69.13%** → 베이스라인 대비 **+1.75%p** (최종 제출 모델)
- **Exp4 (GloVe300 + Sweep):** Test 62.00% → 베이스라인 대비 -5.38%p
- **Exp5 (MiniLM + Sweep):** Test 66.07% → 베이스라인 대비 -1.31%p
- **Exp6 (MPNet + Sweep):** Test 66.28% → 베이스라인 대비 -1.10%p

베이스라인을 넘긴 실험은 Exp2와 Exp3 두 가지로, 다섯 실험 중에서는 TF-IDF를 적용한 Exp3가 가장 좋은 성능을 보였다. 가장 성능이 좋았던 Exp3를 개선하기 위해서 추가적으로 전처리와 핸드크래프트 피처를 더한 추가 개선 버전 **Exp3-v8(Test 69.13%)** 이 최종 제출 모델이다.

실험 구성

```

├─ submit/                                     # 최종 제출 파일
│   └─ W5_HW1_PracticalNLP_202200274_고민서.ipynb # HW1 제출본 (Exp3-v8, 69.13% 🏆)
│       └─ W5_HW2_PracticalNLP_202200274_고민서.ipynb # HW2 제출본 (감성 예측 구현)
├─ notebooks/
│   └─ Exp1_BoW_Baseline.ipynb                # 베이스라인 (67.38%)
│   └─ Exp2_BoW_Sweep.ipynb                  # BoW + Sweep (68.26%)
│   └─ Exp3_TFIDF_Sweep.ipynb                # TF-IDF + Sweep (68.30%)
│   └─ Exp4_GloVe100_Sweep.ipynb             # GloVe 100d
│   └─ Exp4_GloVe300_Sweep.ipynb             # GloVe 300d (62.00%)
│   └─ Exp5_MiniLM_Sweep.ipynb               # MiniLM 384d (66.07%)
│   └─ Exp5_v2_MiniLM_Fixed.ipynb            # MiniLM (seed 고정)
│   └─ Exp6_MPNet_Sweep.ipynb                # MPNet 768d (66.28%)
│   └─ exp2_improvements/
│       └─ Exp2_v2_BoW_Enhanced_Sweep.ipynb # BoW 추가 개선 sweep
│       └─ exp3_improvements/
│           └─ Exp3_v6_Preprocessing.ipynb   # 백틱/슬랭 전처리 (68.36%)
│           └─ Exp3_v7_Handcraft.ipynb       # 핸드크래프트 피처 (68.61%)
│           └─ Exp3_v9_Scaled.ipynb          # StandardScaler 시도 (68.76%)
│           └─ Exp3_v10_Final.ipynb          # v8 config + epoch 100 (69.13%)
└─ analysis/

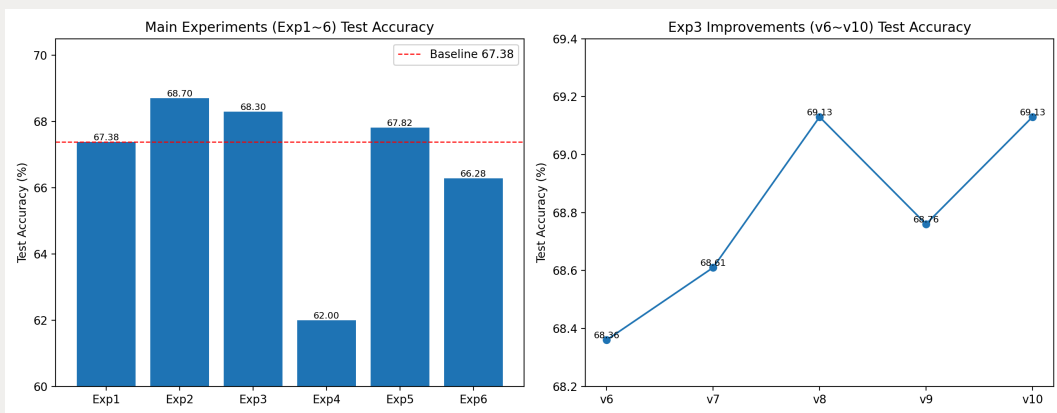
```

```

├── analyze_data.py           # 데이터 기초 분석
├── analyze_data_deep.py      # 데이터 심층 분석
├── docs/
│   ├── HW1_Report_Draft.md  # 실험 보고서
│   ├── 실험_결과_정리.md     # 상세 실험 기록
│   ├── HW1_Results_Summary.ipynb
│   ├── HW2_AllModels.ipynb
│   └── HW2_PracticalNLP.ipynb
├── README.md
└── .gitignore

```

전체 실험 성능 비교 그래프



5. Exp 1: 베이스라인 확인

```

# === MLP 모델 정의: 과제에서 주어진 베이스 구조 그대로 사용 (이후 모든 실험에서 동일
# 구조, 변경 없음) ===
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(MLP, self).__init__()
        # 첫번째 은닉층
        self.fc1 = nn.Linear(input_size, hidden_size)
        # 두번째 은닉층
        self.fc2 = nn.Linear(hidden_size, hidden_size//2)
        # 출력층
        self.fc3 = nn.Linear(hidden_size//2, output_size)
        # 활성화 함수
        self.activation = nn.GELU()
        # 출력층의 활성화 함수
        self.output_activation = nn.Softmax(dim=1)

    def forward(self, x):
        # 입력 x(벡터, 숫자)를 모델에 적용

```

```

out1 = self.fc1(x)
out2 = self.activation(out1)
out3 = self.fc2(out2)
out4 = self.activation(out3)
out5 = self.fc3(out4)
out6 = self.output_activation(out5)
final_out = out6
return final_out

# 하이퍼파라미터 셋팅
input_size = len(vectorizer.vocabulary_) # input size
hidden_size = 1000 # hidden size
output_size = 3 # output size is 3 (positive/negative/neutral)
learning_rate = 0.0001
batch_size = 128
num_epochs = 10
loss_function = nn.CrossEntropyLoss()

```

베이스라인 코드 실행 결과 Test Accuracy

```

38 print(f"Epoch {epoch+1}, Accuracy: {dev_accuracy}, loss: {epoch_loss/len(train_t
Epoch 1, Accuracy: 0.6643611911623439, loss: 3.019593785837537e-05
Epoch 2, Accuracy: 0.684149855907781, loss: 2.685003745995584e-05
Epoch 3, Accuracy: 0.6826128722382324, loss: 2.4611630370138123e-05
Epoch 4, Accuracy: 0.6799231508165225, loss: 2.3268445654482137e-05
Epoch 5, Accuracy: 0.6854947166186359, loss: 2.213648031847399e-05
Epoch 6, Accuracy: 0.6806916426512968, loss: 2.1556728198880056e-05
Epoch 7, Accuracy: 0.6747358309317963, loss: 2.142034669513585e-05
Epoch 8, Accuracy: 0.6737752161383286, loss: 2.1705188269375777e-05
Epoch 9, Accuracy: 0.6726224783861672, loss: 2.0989667257813156e-05
Epoch 10, Accuracy: 0.6662824207492796, loss: 2.1264191762712157e-05

1 ##### 테스트 세트로 모델 평가하기
2 from sklearn.metrics import accuracy_score # Accuracy 측정 함수 import
3
4 test_tensors = torch.cuda.FloatTensor(test_vectors.toarray(), device=device)
5 pred_results = best_model(test_tensors) # 최종 모델로 test 데이터 예측
6 pred_labels = torch.argmax(pred_results, axis=1)
7
8 accuracy = accuracy_score(test_data['label'], pred_labels.tolist()) # 정확도 측정
9 print("Accuracy: {:.2f}%".format(accuracy*100)) # 정확도 출력

... Accuracy: 67.38%

```

가장 먼저 한 일은 베이스라인 코드를 그대로 돌려서 재현 가능한 기준점을 확보하는 것이었다. CountVectorizer 기반의 BoW 표현에 은닉층 크기 1000, 학습률 1e-4, 배치 크기 128, 10 에폭으로 학습한 결과, dev에서 68.55%, test에서 67.38%가 나왔다. 이후 모든 비교는 이 67.38%를 기준선으로 삼았다.

6. Exp 2: 베이스라인 코드 BoW+자동 튜닝

Exp2에서는 벡터화 방법은 그대로 둔 채 W&B Sweep으로 학습률, 은닉층 크기, 드롭아웃, weight decay, 배치 크기, 에폭 수를 함께 탐색했다. Bayesian 방식으로 20회 시도한 결과, 은닉층 크기 512, 학습률 약 6.35e-5, 드롭아웃 0.1, weight decay 1e-4, 배치 크기 256, 50 에폭이라는 조합이 최적으로 선택되었다. 이 설정으로 dev 0.6945, test 68.26%를 얻어 베이스라인 대비 0.88%p 향상시킬 수 있었다.

아래 코드 블록의 앞부분(MLP 모델 정의, sweep용 학습 함수 팩토리, 기본 SWEEP_CFG)은 Exp2부터 Exp6까지 다섯 모델에서 동일하게 적용한 공통 코드이고, 각 실험에서 달라지는 부분만 그 뒤에 따로 적었다. Exp2의 고유한 부분은 BoW 입력을 그대로 쓰는 것 한 가지뿐이라, 베이스라인에서 만든 `train_t/dev_t/test_t`를 재사용해 sweep만 실행했다.

```
# =====
# [공통 코드] Exp2 ~ Exp6 모두 동일하게 적용 (한 번만 정의해 재사용)
# =====

# === 공통 1) MLP 모델 정의 - 과제 제약상 구조 변경 없음, 입력 차원/은닉 크기/드롭아웃만 인자로 받음 ===
class MLP(nn.Module):
    def __init__(self, i, h, o, d=0.0):
        super().__init__()
        self.fc1=nn.Linear(i,h)
        self.fc2=nn.Linear(h,h//2)
        self.fc3=nn.Linear(h//2,o)
        self.activation=nn.GELU()
        self.output_act=nn.Softmax(dim=1)
        self.dropout=nn.Dropout(p=d)
    def forward(self,x):
        x=self.dropout(self.activation(self.fc1(x)))
        x=self.dropout(self.activation(self.fc2(x)))
        return self.output_act(self.fc3(x))

# === 공통 2) Sweep용 학습 함수 팩토리 - wandb run마다 학습/평가, best dev 가중치로 test 측정 ===
def make_sweep_fn(train_t,dev_t,dev_l,test_t,inp,lbl):
    def train_fn():
        with wandb.init() as run:
            cfg=run.config
            torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
            model=MLP(inp,cfg.hidden_size,output_size,cfg.dropout).to(device)

            opt=optim.Adam(model.parameters(),lr=cfg.learning_rate,weight_decay=cfg.weight_decay)
            lfn=nn.CrossEntropyLoss()
            best_dev,best_state=0,None
            for epoch in range(cfg.num_epochs):
                model.train()
                eloss=0
                for i in range(0,len(train_t),cfg.batch_size):
                    bd=train_t[i:i+cfg.batch_size]
                    bl=torch.tensor(train_labels[i:i+cfg.batch_size],device
```

```

e=device)

        out=model(bd)
        loss=lnf(out,bl)
        opt.zero_grad(); loss.backward(); opt.step()
        eloss+=loss.item()
    model.eval()
    with torch.no_grad():
        da=(torch.argmax(model(dev_t),dim=1)==dev_l).float().mean().item()

        if da>best_dev:
            best_dev=da
            best_state=copy.deepcopy(model.state_dict())
            wandb.log({'epoch':epoch+1,'dev_accuracy':da,'best_dev_accuracy':best_dev,'train_loss':eloss/len(train_t)})
            model.load_state_dict(best_state)
            model.eval()
            with torch.no_grad():
                tp=torch.argmax(model(test_t),dim=1)
                ta=accuracy_score(test_labels_list,tp.cpu().tolist())
            wandb.log({'test_accuracy':ta})
            print(f'[{lbl}] Dev:{best_dev:.4f}|Test:{ta*100:.2f}%')
    return train_fn

# === 공통 3) 기본 SWEEP_CFG - Exp2/3/4/6 그대로 사용, Exp5만 별도 SWEEP_CFG_MINILM으로 재정의 ===
SWEEP_CFG={'method':'bayes','metric':{'name':'best_dev_accuracy','goal':'maximize'},
'parameters':{'learning_rate':{'distribution':'log_uniform_values','min':1e-5,'max':1e-2},
'hidden_size':{'values':[256,512,1000,2000]},'dropout':{'values':[0.0,0.1,0.2,0.3,0.5]},
'weight_decay':{'values':[0.0,1e-5,1e-4,1e-3]},'num_epochs':{'values':[20,30,50]},
'batch_size':{'values':[64,128,256]}}}

# =====
# [Exp2 고유] BoW 입력은 베이스라인에서 만든 train_t/dev_t/test_t를 그대로 사용
# =====
sweep_id=wandb.sweep(**SWEEP_CFG,'name':'exp2-bow',project='nlp-hw1')
print(f'Sweep ID:{sweep_id}')
train_fn=make_sweep_fn(train_t,dev_t,dev_labels_t,test_t,input_size,'Exp2-BoW')
wandb.agent(sweep_id,function=train_fn,count=20)

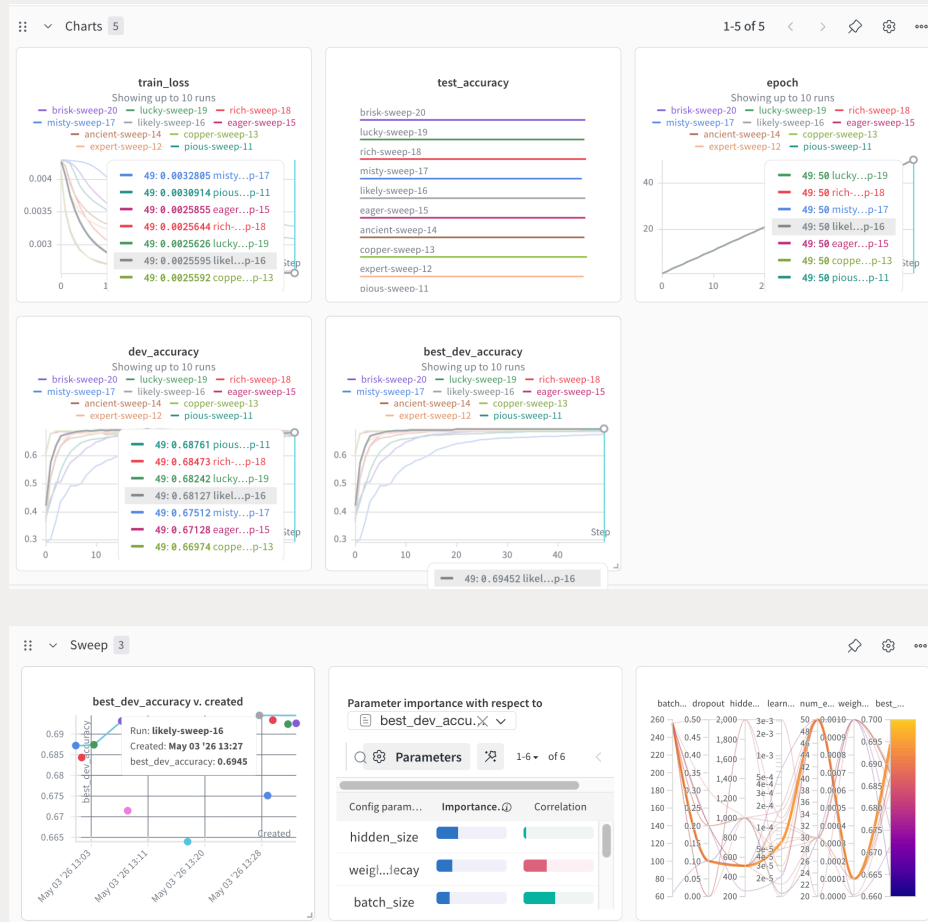
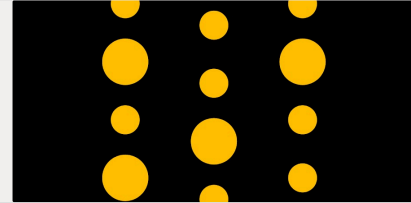
```

Figure 1. Exp2 W&B Sweep 결과 — BoW + 자동 탐색. Bayesian 탐색 20회의 best dev accuracy 추이와 학습률·은닉층·드롭아웃·weight decay·배치 크기·에폭 등 6개 하이퍼파라미터별 분포가 함께 담겨 있다.

imeanseo_

Weights & Biases, developer tools for machine learning

🔗 https://wandb.ai/imeanseo_/nlp-hw1/sweeps/gzg4mb69?nw=nwuserailee_n02ko



Run data is saved locally in /content/wandb/run-20260505_053804-dm65b70
 Syncing run **noble-sweep-19** to **Weights & Biases (docs)**
 Sweep page: <https://wandb.ai/imeanseo/nlp-hw1/sweeps/jjeov1va>
 View project at <https://wandb.ai/imeanseo/nlp-hw1>
 View sweep at <https://wandb.ai/imeanseo/nlp-hw1/sweeps/jjeov1va>
 View run at <https://wandb.ai/imeanseo/nlp-hw1/runs/dm56b70>
 [Exp3-TF-IDF] Dev: 0.6924 | Test: 68.49%

```
best_dev_accuracy 0.69241
dev_accuracy      0.69241
epoch             50
```

```
... wandb: Sorting runs by -summary_metrics.best_dev_accuracy
```

Best Dev:0.6934
Test:68.30%

✅ 저장:best_model_exp3.pt|Dev:0.6934|Test:68.30%

9

흥미로웠던 점은 은닉층을 베이스라인의 절반(1000 → 512)으로 줄이면서도 드롭아웃은 0.1로 가볍게만 걸고 weight decay 1e-4를 살짝 더하는 정도가 최적으로 잡혔다는 점이다. 강한 정규화를 거는 대신 모델 크기를 적당히 줄이고 학습률을 다소 낮춰 천천히 수렴시키는 균형이 이 데이터에 더 잘 맞았다. 데이터의 신호 자체가 단순한 키워드 패턴 위주이기 때문에, 모델 용량을 키우기보다 적절한 크기에서 가벼운 정규화로 학습 안정성을 잡아주는 쪽이 일반화에 유리했던 것으로 해석된다.

7. Exp 3: 표현 방법 TF-IDF로 변경

Exp3는 BoW를 TF-IDF로 교체하고 Exp2와 같은 sweep 설정을 다시 돌린 실험이다. test에서 68.30%로 가장 높은 성능이 나왔고 베이스라인 대비 0.92%p 향상이라는 결과를 얻었다. 단어의 단순 빈도 대신 문서 빈도의 역수까지 반영하기 때문에 "good"이나 "bad"처럼 감성 신호가 강한 단어의 가중치가 자연스럽게 커지는 점이 이 데이터셋에 잘 맞은 것으로 보인다.

공통 코드(MLP 모델 정의, `make_sweep_fn`, 기본 `SWEEP_CFG`)는 Exp2와 동일하게 적용했고, Exp3에서 달라진 부분은 입력 표현을 BoW에서 TF-IDF로 교체한 것뿐이다.

```
# =====
# [Exp3 고유] 입력 표현을 BoW → TF-IDF로 교체 (max_features=30000)
# 공통 코드(MLP / make_sweep_fn / SWEEP_CFG)는 Exp2와 동일하게 적용
# =====
vectorizer=TfidfVectorizer(max_features=30000)
vectorizer.fit(train_data['text']) # train에만 fit (data leakage 방지)
train_t=torch.FloatTensor(vectorizer.transform(train_data['text']).toarray()).to(device)
dev_t =torch.FloatTensor(vectorizer.transform(dev_data['text']).toarray()).to(device)
test_t =torch.FloatTensor(vectorizer.transform(test_data['text']).toarray()).to(device)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=train_t.shape[1] # 실제 어휘 수는 30000보다 작을 수 있음
print(f'TF-IDF vocab:{input_size}')

# 공통 SWEEP_CFG 그대로 사용
sweep_id=wandb.sweep({**SWEEP_CFG, 'name': 'exp3-tfidf'}, project='nlp-hw1')
train_fn=make_sweep_fn(train_t, dev_t, dev_labels_t, test_t, input_size, 'Exp3-TFIDF')
wandb.agent(sweep_id, function=train_fn, count=20)
```



Figure 2. Exp3 TF-IDF Sweep 학습 결과. Best Dev 69.34%, Test 68.30%로 다섯 번 실험 중 가장 높은 성능을 기록한 run의 학습 곡선이다.

```

9 print(f"\nBest Dev:{best.summary['best_dev_accuracy']:.4f}")
10 print(f"Test:{best.summary['test_accuracy']*100:.2f}%")
11 print('='*60)

wandb: Sorting runs by --summary_metrics.best_dev_accuracy
=====
🏆 Best Run Config:
dropout           : 0.3
batch_size        : 256
num_epochs        : 50
hidden_size       : 256
weight_decay      : 0
learning_rate     : 5.591937393848229e-05

Best Dev:0.6934
Test:68.30%
=====

Best Config로 재학습 + 저장

1 # 🏆 위 출력 값으로 수정
2 BEST_H,BEST_LR,BEST_D,BEST_WD,BEST_EP,BEST_BS=256,5.591937393848229e-05,0.3,0,50,256
3 torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
4 final=MLP(input_size,BEST_H,output_size,BEST_D).to(device)
5 opt=optim.Adam(final.parameters(),lr=BEST_LR,weight_decay=BEST_WD)
6 lfn=nn.CrossEntropyLoss()

Epoch 22/50 | Dev:0.6932
Epoch 23/50 | Dev:0.6903
... Epoch 24/50 | Dev:0.6895
Epoch 25/50 | Dev:0.6888
Epoch 26/50 | Dev:0.6874
Epoch 27/50 | Dev:0.6861
Epoch 28/50 | Dev:0.6866
Epoch 29/50 | Dev:0.6870
Epoch 30/50 | Dev:0.6861
Epoch 31/50 | Dev:0.6836
Epoch 32/50 | Dev:0.6832
Epoch 33/50 | Dev:0.6849
Epoch 34/50 | Dev:0.6840
Epoch 35/50 | Dev:0.6832
Epoch 36/50 | Dev:0.6822
Epoch 37/50 | Dev:0.6836
Epoch 38/50 | Dev:0.6840
Epoch 39/50 | Dev:0.6826
Epoch 40/50 | Dev:0.6805
Epoch 41/50 | Dev:0.6795
Epoch 42/50 | Dev:0.6797
Epoch 43/50 | Dev:0.6799
Epoch 44/50 | Dev:0.6803
Epoch 45/50 | Dev:0.6797
Epoch 46/50 | Dev:0.6778
Epoch 47/50 | Dev:0.6763
Epoch 48/50 | Dev:0.6763
Epoch 49/50 | Dev:0.6742
Epoch 50/50 | Dev:0.6742

✅ 저장: best_model_exp3.pt | Dev:0.6934 | Test:68.30%

```

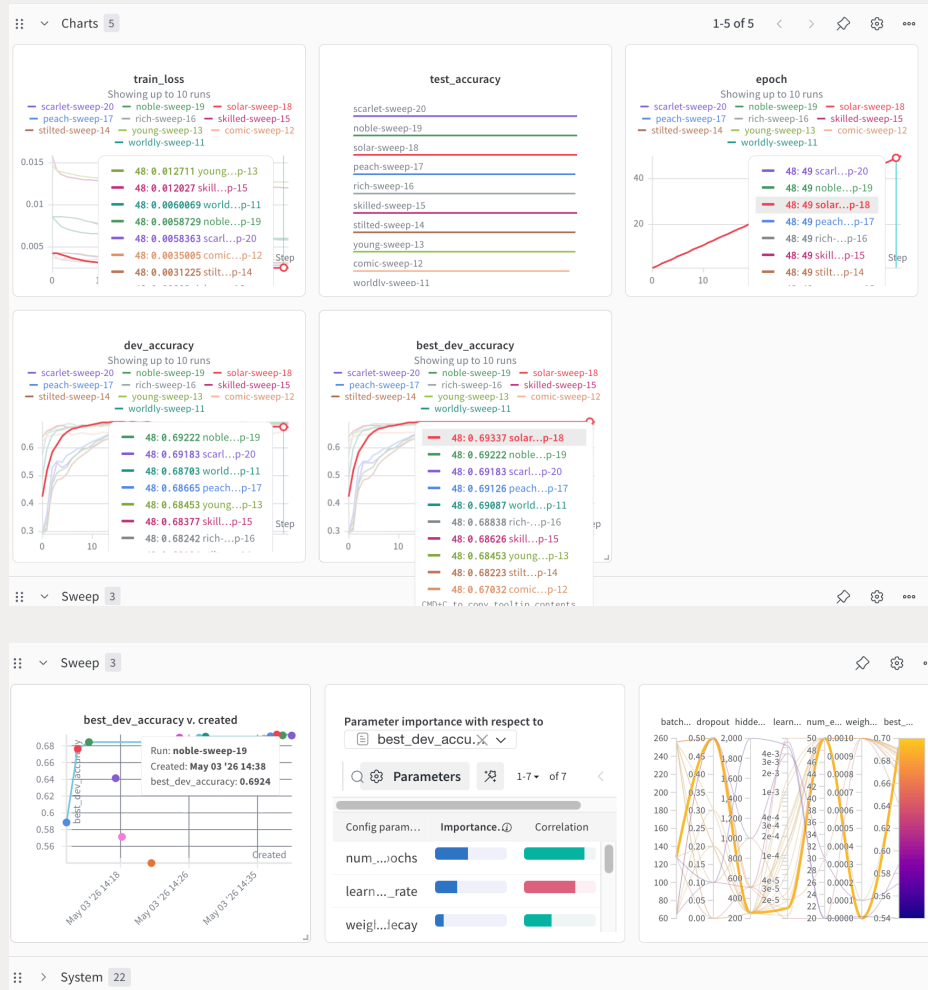
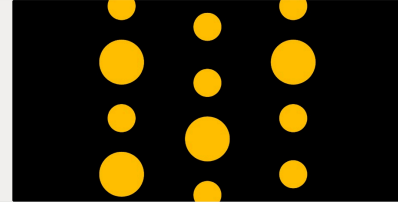


Figure 3. Exp3 Sweep 최적 하이퍼파라미터. Bayesian 탐색이 최종적으로 선택한 best config(은닉층·학습률·드롭아웃·weight decay·배치 크기·에폭) 화면이다.

imeanseor_

Weights & Biases, developer tools for machine learning

https://wandb.ai/imeanseor_/nlp-hw1/sweeps/jjeov1va?nw=nwuserai-leen02ko



주목할 점은 BoW에 sweep만 적용한 Exp2가 이미 68.26%까지 올라와 있었기 때문에, TF-IDF로 표현만 바꾸는 한 걸음은 그 위에 +0.04%p의 작은 추가 향상에 그쳤다는 것이다. 표현 변경 자체로 큰 점프가 일어난 것이 아니라, sweep이 만들어낸 안정된 학습 토대 위에 TF-IDF가 한 단계 더 얹혀지는 형태에 가까웠다. 그래도 결과가 안정적이고 재현 가능했고, 이후 세부 조정을 올리기에 가장 적합한 표현이라고 판단해 이 설정을 추가 개선 실험(v6~v10)의 출발점으로 삼았고, 그중 가장 높은 성능을 낸 v8 설정이 최종 제출 모델이 되었다.

8. Exp 4: 분산 표현을 임베딩(GloVe300)으로 바꿔보기

Exp4에서는 사전학습된 GloVe 300차원 임베딩을 사용했다. 각 단어를 300차원 벡터로 바꾼 뒤 문장 안의 단어 벡터들을 평균 내어 MLP에 넣는 방식이다. sweep으로 찾은 best 설정(은닉층 크기 2000, 학습률 약 $9.64e-4$,

드롭아웃 0, weight decay 0, 배치 크기 256, 50 에폭)으로 다시 학습시킨 결과 Dev는 Epoch 1의 0.6037에서 천천히 올라와 Epoch 17쯤 0.635 부근에 도달한 뒤 Epoch 37에서 0.6382의 고점을 찍었고, 이후로는 0.62~0.64 사이를 진동하며 마무리됐다. best_state로 저장된 시점의 Test는 62.00%로 베이스라인보다 5.38%p 낮았다. 다른 실험들과 달리 정규화를 거의 걸지 않은 큰 모델에 비교적 큰 학습률이 best로 잡혔다는 점이 특이했는데, 입력 차원이 300으로 작아 모델 용량을 키워도 과적합 부담이 적었고 학습률을 비교적 크게 가져가야 학습이 충분히 진행되었기 때문으로 보인다.

공통 코드(MLP, `make_sweep_fn`, 기본 `SWEEP_CFG`)는 Exp2와 동일하게 적용했고, Exp4에서 달라진 부분은 입력 표현을 GloVe 300d 평균 풀링으로 교체한 것이다.

```
# =====
# [Exp4 고유] 입력 표현을 GloVe 300d 평균 풀링으로 교체
# 공통 코드(MLP / make_sweep_fn / SWEEP_CFG)는 Exp2와 동일하게 적용
# =====
import gensim.downloader as gensim_api, re
glove=gensim_api.load('glove-wiki-gigaword-300')

# 문장 → 단일 벡터: 단어별 300d 벡터를 모은 뒤 평균 풀링으로 압축
def texts_to_glove(texts,model,dim=300):
    vecs=[]
    for sent in texts:
        words=re.sub(r'[^a-zA-Z\s]','',sent.lower()).split() # 구두점 제거
        + 소문자화
        ww=[model[w] for w in words if w in model] # 00V 단어는
        자동 제외
        vecs.append(np.mean(ww,axis=0) if ww else np.zeros(dim))
    return np.array(vecs,dtype=np.float32)

train_t=torch.FloatTensor(texts_to_glove(train_data['text'],glove)).to(device)
dev_t =torch.FloatTensor(texts_to_glove(dev_data['text'],glove)).to(device)
test_t =torch.FloatTensor(texts_to_glove(test_data['text'],glove)).to(device)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=300

# 공통 SWEEP_CFG 그대로 사용
sweep_id=wandb.sweep({'**SWEEP_CFG','name':'exp4-glove'},project='nlp-hw1')
train_fn=make_sweep_fn(train_t,dev_t,dev_labels_t,test_t,input_size,'Exp4-GloVe')
wandb.agent(sweep_id,function=train_fn,count=20)
```



Figure 4. Exp4 GloVe300 학습 결과. sweep best config로 50 에폭 재학습한 곡선으로, Epoch 37에서 Dev 0.6382의 고점을 찍은 뒤 0.62~0.64 사이를 진동하다가 Test 62.00%로 마무리됐다.

```

1 USERNAME='imeanse_'
2 api=wandb.Api()
3 sw=api.sweep(f'{USERNAME}/nlp-hw1/{sweep_id}')
4 best=sw.best_run()
5 print('\n'+ '='*60)
6 print('🏆 Best Run Config:')
7 for k,v in dict(best.config).items():
8     print(f'   {k:<20}: {v}')
9 print(f"\nBest Dev:{best.summary['best_dev_accuracy']:.4f}")
10 print(f"Test:{best.summary['test_accuracy']*100:.2f}%")
11 print('='*60)

```

wandb: Sorting runs by -summary_metrics.best_dev_accuracy

=====

🏆 Best Run Config:

dropout	: 0.1
batch_size	: 256
num_epochs	: 30
hidden_size	: 1000
weight_decay	: 1e-05
learning_rate	: 0.004858558426734933

Best Dev:0.5946

Test:58.48%

=====

```

Epoch 30/50|Dev:0.622/
Epoch 31/50|Dev:0.6359
... Epoch 32/50|Dev:0.6300
Epoch 33/50|Dev:0.6330
Epoch 34/50|Dev:0.6340
Epoch 35/50|Dev:0.6311
Epoch 36/50|Dev:0.6280
Epoch 37/50|Dev:0.6382
Epoch 38/50|Dev:0.6271
Epoch 39/50|Dev:0.6350
Epoch 40/50|Dev:0.6277
Epoch 41/50|Dev:0.6300
Epoch 42/50|Dev:0.6259
Epoch 43/50|Dev:0.6292
Epoch 44/50|Dev:0.6298
Epoch 45/50|Dev:0.6319
Epoch 46/50|Dev:0.6290
Epoch 47/50|Dev:0.6248
Epoch 48/50|Dev:0.6206
Epoch 49/50|Dev:0.6236
Epoch 50/50|Dev:0.6284

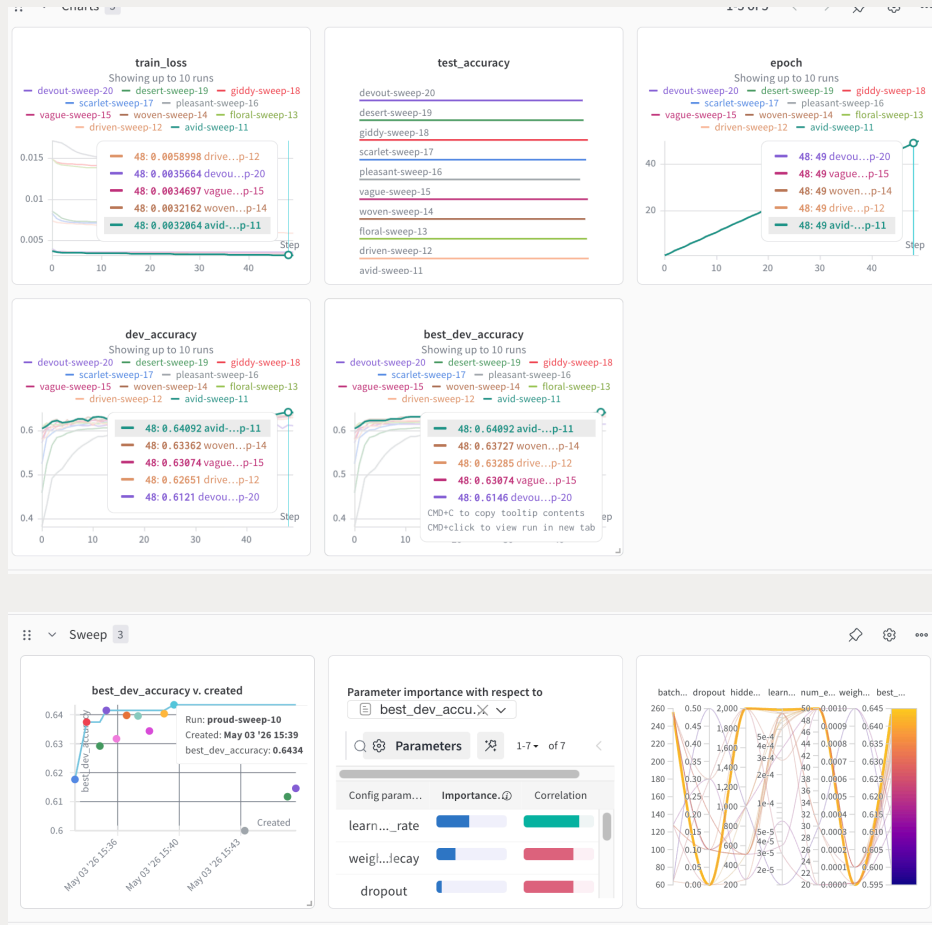
```

✅ 저장:best_model_exp4.pt|Dev:0.6382|Test:62.00%

imeanseor

Weights & Biases, developer tools for machine learning

<https://wandb.ai/imeanseor/nlp-hw1/sweeps/t8m6rpyj?nwuser=aileen02ko>



원인을 살펴보면 평균이라는 연산 자체가 단어 순서와 부정 표현을 모두 흘려버린다는 점이 컸다. 예를 들어 "not good"이라는 표현에서도 "good"의 긍정 벡터가 그대로 평균에 들어가 부정의 의미가 희석된다. 수업에서 다른 시퀀스를 그대로 펼쳐 9000차원 입력으로 쓰는 방식과 달리 과제에서는 평균으로 압축해야 했기 때문에 GloVe의 장점을 살리기 어려웠다. 학습률 범위를 낮추고 구두점 제거 전처리를 추가하는 등 보완을 시도했지만, 평균화의 근본적 한계는 해결되지 않았다.

9. Exp5, 6: 문맥 임베딩을 시도한 MiniLM과 MPNet

Exp5에서는 sentence-transformers의 MiniLM(384차원)을 문장 임베딩으로 사용했다. sweep으로 찾은 best 설정으로 다시 학습시킨 결과 Dev는 Epoch 3에서 0.6730의 고점을 찍은 뒤로 50 에폭 내내 회복하지 못했고, best_state로 저장된 Epoch 3 모델의 Test는 66.07%로 베이스라인보다 1.31%p 낮았다. 학습 곡선이 초반에 한 번 올라간 뒤로는 미세하게 진동하면서 천천히 내려가는 모양이라, MiniLM 임베딩 위에서는 이 데이터에 맞는 학습 신호가 길게 이어지지 못한다는 인상을 받았다.

공통 코드(MLP, `make_sweep_fn`)는 Exp2와 동일하게 적용했고, Exp5에서는 입력 표현을 MiniLM(384d) 문장 임베딩으로 교체했다. 사전학습 임베딩 위에서는 학습률 스케일과 모델 용량의 적정 영역이 좁기 때문에 기본

SWEEP_CFG보다 탐색 공간을 더 좁게 재정의한 `SWEEP_CFG_MINILM` 을 사용했다.

```
# =====
# [Exp5 고유] 입력 표현을 MiniLM(384d) 문장 임베딩으로 교체
# 공통 코드 중 MLP / make_sweep_fn 은 Exp2와 동일, SWEEP_CFG만 별도 정의
# =====
from sentence_transformers import SentenceTransformer
model_emb=SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
# encode()가 토큰나이징 → forward → mean pooling을 한 번에 처리해 384d 문장 벡터를 바로 반환 (GloVe처럼 따로 평균낼 필요 없음)
train_t=torch.FloatTensor(model_emb.encode(train_data['text'],batch_size=128,convert_to_numpy=True)).to(device)
dev_t =torch.FloatTensor(model_emb.encode(dev_data['text'],batch_size=128,convert_to_numpy=True)).to(device)
test_t =torch.FloatTensor(model_emb.encode(test_data['text'],batch_size=128,convert_to_numpy=True)).to(device)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=384 # MiniLM 출력 차원

# === Exp5 고유 SWEEP_CFG: 사전학습 임베딩에 맞춰 lr 상한↓, hidden 256 제외, dropout/wd/epochs/batch 탐색 범위도 줄임 ===
SWEEP_CFG_MINILM={'method':'bayes','metric':{'name':'best_dev_accuracy','goal':'maximize'},
'parameters':{'learning_rate':{'distribution':'log_uniform_values','min':1e-5,'max':1e-3},
'hidden_size':{'values':[512,1000,2000]},'dropout':{'values':[0.0,0.1,0.2,0.3]},
'weight_decay':{'values':[0.0,1e-5,1e-4]},'num_epochs':{'values':[30,50]},
'batch_size':{'values':[128,256]}}}

sweep_id=wandb.sweep(**SWEEP_CFG_MINILM,'name':'exp5-minilm',project='nlp-hw1')
train_fn=make_sweep_fn(train_t,dev_t,dev_labels_t,test_t,input_size,'Exp5-MiniLM')
wandb.agent(sweep_id,function=train_fn,count=20)
```




Figure 5. Exp5 MiniLM 학습 결과. sweep best config 재학습 결과로, Dev가 Epoch 3에서 0.6730의 고점을 찍은 뒤 50 에폭 내내 회복하지 못한 학습 곡선이 그대로 드러난다(Test 66.07%).

```

4 best=sw.best_run()
5 print('\n'+'*60)
6 print('🏆 Best Run Config:')
7 for k,v in dict(best.config).items():
8     print(f' {k:<20}: {v}')
9 print(f"\nBest Dev:{best.summary['best_dev_accuracy']:.4f}")
10 print(f"Test:{best.summary['test_accuracy']*100:.2f}%")
11 print('='*60)

```

... wandb: Sorting runs by -summary_metrics.best_dev_accuracy

```

=====
🏆 Best Run Config:
dropout           : 0
batch_size        : 256
num_epochs        : 50
hidden_size       : 2000
weight_decay      : 1e-05
learning_rate     : 0.0002574056014229813

```

```

Best Dev:0.6730
Test:66.07%
=====

```

Best Config 재학습 - 결과

```

Epoch 30/50|Dev:0.6596
Epoch 31/50|Dev:0.6580
Epoch 32/50|Dev:0.6569
Epoch 33/50|Dev:0.6586
Epoch 34/50|Dev:0.6607
Epoch 35/50|Dev:0.6611
Epoch 36/50|Dev:0.6599
Epoch 37/50|Dev:0.6609
Epoch 38/50|Dev:0.6628
Epoch 39/50|Dev:0.6640
Epoch 40/50|Dev:0.6636
Epoch 41/50|Dev:0.6671
Epoch 42/50|Dev:0.6659
Epoch 43/50|Dev:0.6667
Epoch 44/50|Dev:0.6665
Epoch 45/50|Dev:0.6672
Epoch 46/50|Dev:0.6682
Epoch 47/50|Dev:0.6676
Epoch 48/50|Dev:0.6678
Epoch 49/50|Dev:0.6671
Epoch 50/50|Dev:0.6680

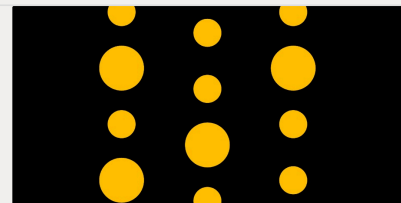
```

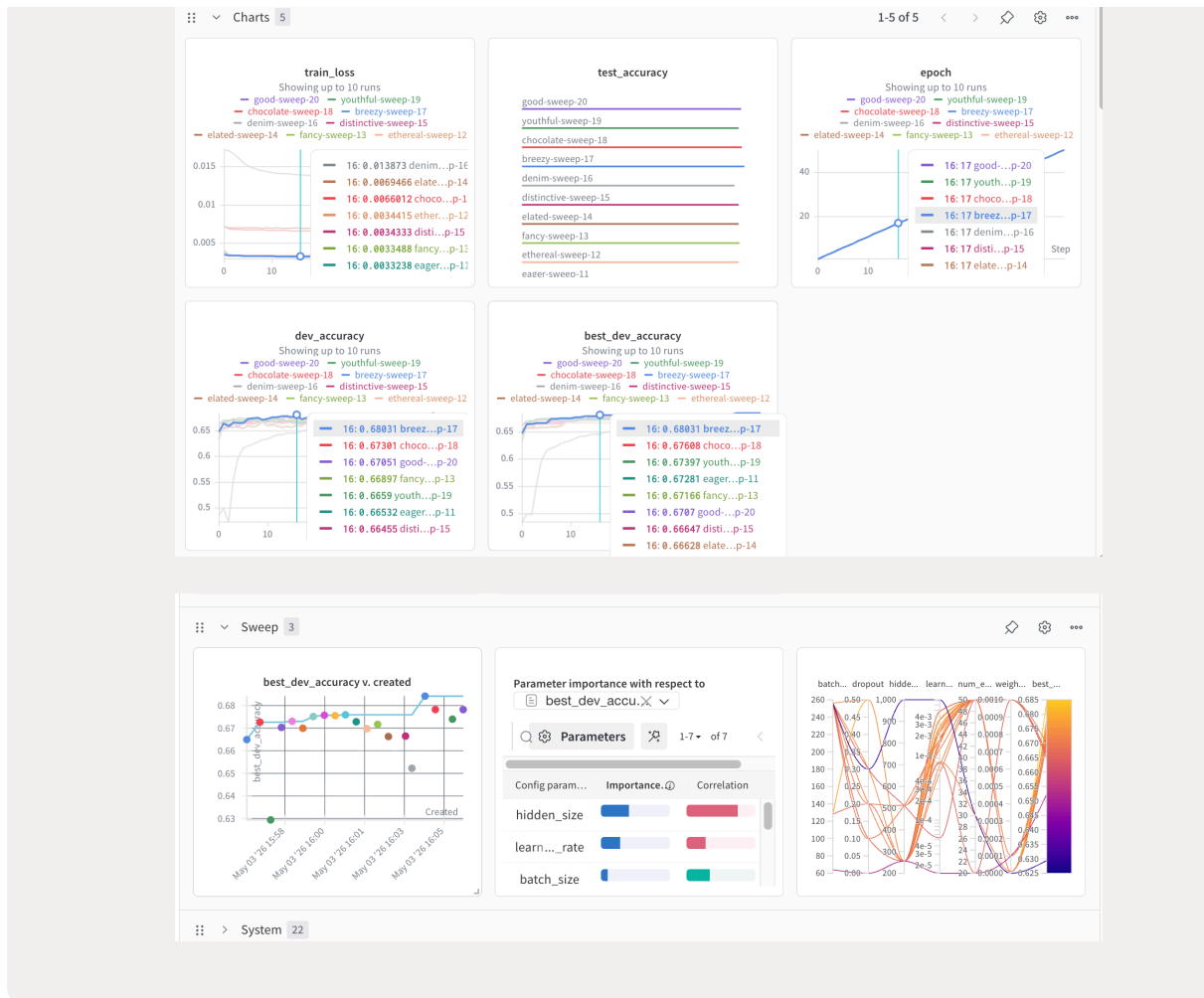
✅ 저장:best_model_exp5.pt|Dev:0.6730|Test:66.07%

imeanseos_

Weights & Biases, developer tools for machine learning

🔗 <https://wandb.ai/imeanseos/nlp-hw1/sweeps/kt6o002w?nw=nwuse-raileen02ko>





Exp6에서는 같은 계열의 더 큰 모델인 MPNet(768차원)으로 바꿔 보았다. sweep으로 찾은 best 설정으로 다시 학습시킨 결과 Dev는 50 에폭 내내 0.66~0.68 사이에서 진동하며 Epoch 39에서 0.6788의 고점을 찍었고, best_state로 저장된 시점의 Test는 66.28%로 베이스라인보다 1.10%p 낮았다. MiniLM과 마찬가지로 사전학습된 임베딩 위에 MLP를 얹는 구조에서는 학습률을 작게 가져가야 안정적이지만, 너무 낮으면 학습이 충분히 진행되지 않는 균형점을 sweep으로도 깔끔하게 잡기 어려웠다. 곡선이 우상향하는 추세와 큰 진동이 계속 섞여 나타난 것도 이 불안정성을 보여주는 부분이다. 또한 768차원의 풍부한 표현을 단순한 MLP로 압축하다 보니 문맥 임베딩 본래의 장점이 충분히 발휘되지 못한 측면도 있다.

공통 코드(MLP, `make_sweep_fn`, 기본 `SWEEP_CFG`)는 Exp2와 동일하게 적용했다. Exp6에서는 입력 표현을 같은 sentence-transformers 계열의 더 큰 MPNet(768d)으로 바꿨고, 모델이 커진 만큼 인코딩 `batch_size`만 128에서 64로 낮췄으며, `SWEEP_CFG`는 Exp5와 달리 다시 기본 설정을 그대로 사용했다.

```
# =====
# [Exp6 고유] 입력 표현을 MPNet(768d) 문장 임베딩으로 교체 (MiniLM 대비 약 2배 큰
# 출력 차원)
# 공통 코드(MLP / make_sweep_fn / SWEEP_CFG)는 Exp2와 동일하게 적용
# =====
from sentence_transformers import SentenceTransformer
model_emb=SentenceTransformer('sentence-transformers/all-mpnet-base-v2')
# 인터페이스는 MiniLM과 동일, 다만 모델이 커서 인코딩 batch_size를 128 → 64로 낮춤
train_t=torch.FloatTensor(model_emb.encode(train_data['text'],batch_size=64,convert_to_numpy=True)).to(device)
```

```

dev_t =torch.FloatTensor(model_emb.encode(dev_data['text'],batch_size=64,
convert_to_numpy=True)).to(device)
test_t =torch.FloatTensor(model_emb.encode(test_data['text'],batch_size=6
4,convert_to_numpy=True)).to(device)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=768 # MPNet 출력 차원

# 공통 SWEEP_CFG 그대로 사용 (Exp5와 달리 탐색 공간을 다시 넓힘)
sweep_id=wandb.sweep({'**SWEEP_CFG','name':'exp6-mpnet'},project='nlp-hw1')
train_fn=make_sweep_fn(train_t,dev_t,dev_labels_t,test_t,input_size,'Exp6-
MPNet')
wandb.agent(sweep_id,function=train_fn,count=20)

```



Figure 6. Exp6 MPNet 학습 결과. sweep best config 재학습 결과로, 50 에폭 내내 0.66~0.68 사이에서 큰 진동을 보이며 Epoch 39에서 0.6788의 고점을 찍었다(Test 66.28%).

```
9 print(f"\nBest Dev:{best.summary['best_dev_accuracy']:.4f}")
10 print(f"Test:{best.summary['test_accuracy']*100:.2f}%")
11 print('='*60)
```

wandb: Sorting runs by -summary_metrics.best_dev_accuracy

=====

🏆 Best Run Config:

```
dropout          : 0.1
batch_size       : 128
num_epochs       : 50
hidden_size      : 512
weight_decay     : 1e-05
learning_rate    : 0.003965933641576531
```

Best Dev:0.6788

Test:66.28%

=====

```
... Epoch 29/50|Dev:0.6738
Epoch 30/50|Dev:0.6661
Epoch 31/50|Dev:0.6688
Epoch 32/50|Dev:0.6669
Epoch 33/50|Dev:0.6738
Epoch 34/50|Dev:0.6686
Epoch 35/50|Dev:0.6728
Epoch 36/50|Dev:0.6724
Epoch 37/50|Dev:0.6676
Epoch 38/50|Dev:0.6719
Epoch 39/50|Dev:0.6788
Epoch 40/50|Dev:0.6671
Epoch 41/50|Dev:0.6694
Epoch 42/50|Dev:0.6707
Epoch 43/50|Dev:0.6770
Epoch 44/50|Dev:0.6734
Epoch 45/50|Dev:0.6663
Epoch 46/50|Dev:0.6761
Epoch 47/50|Dev:0.6644
Epoch 48/50|Dev:0.6680
Epoch 49/50|Dev:0.6747
Epoch 50/50|Dev:0.6722
```

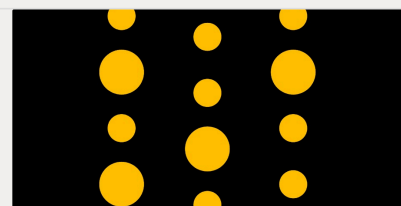
✅ 저장:best_model_exp6.pt|Dev:0.6788|Test:66.28%

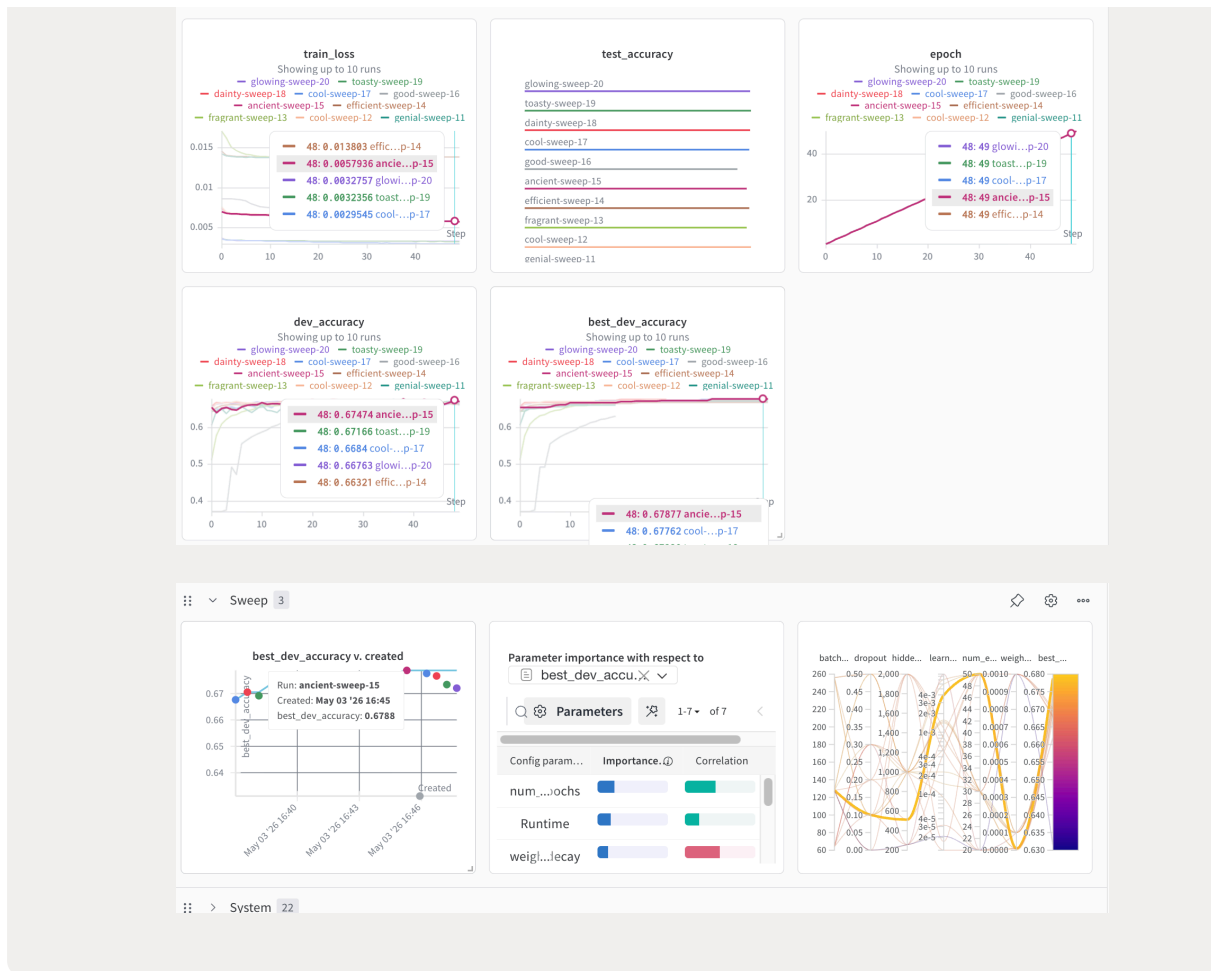
```
1 from google.colab import files
2 files.download('best_model_exp6.pt')
```

imeanseos_

Weights & Biases, developer tools for machine learning

🔗 <https://wandb.ai/imeanseos/nlp-hw1/sweeps/v6x7gb20?nw=nwuse-raileen02ko>





10. 성능이 향상된 실험들의 공통점

베이스라인을 넘긴 두 실험(Exp2, Exp3)을 보면, 둘 다 입력 표현이 BoW·TF-IDF처럼 단어 단위의 명시적인 신호에 가깝다는 공통점이 있다. 특히 가장 좋은 성능을 낸 TF-IDF 실험은 단어의 등장 여부를 넘어 중요도까지 반영하는 단순한 가중치 조정만으로도 의미 있는 향상을 만들어냈다. 이는 이 데이터셋이 결국 핵심 감성 키워드의 존재 여부에 크게 의존하며, 사전학습된 임베딩처럼 더 추상적인 표현이 오히려 이 신호를 흐릴 수 있다는 점을 보여준다.

또 하나의 공통점은 W&B Sweep을 통한 자동 하이퍼파라미터 탐색이다. 사람이 직관적으로 잡기 어려운 학습률·드롭아웃·배치 크기 조합을 Bayesian 탐색으로 빠르게 좁혀 나갔고, 이 과정 자체가 BoW 표현을 그대로 둔 상태에서 베이스라인 대비 약 0.88%p의 향상을 가져왔다.

11. 성능이 떨어진 실험들의 원인 분석

GloVe·MiniLM·MPNet 세 실험에서 성능이 떨어진 데에는 공통적으로 두 가지 원인이 있었다. 하나는 임베딩 차원이 커질수록 평균화 또는 단순 MLP 압축으로 잃어버리는 정보가 많아진다는 점이다. 다른 하나는 사전학습된 임베딩 위에 학습 가능한 MLP를 얹는 구조가, 우리가 익숙한 단순 분류기와는 다른 학습률 스케일을 요구한다는 점이다. 이 두 가지를 동시에 해결하려면 모델 구조 자체를 손봐야 했지만, 과제 제약상 MLP를 그대로 유지해야 했기에 한계가 분명했다.

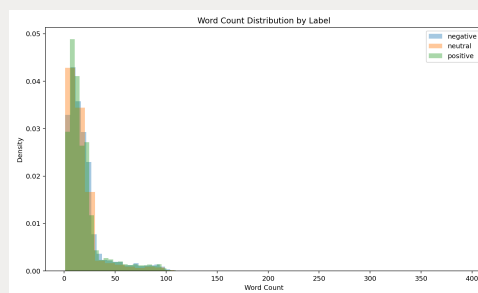
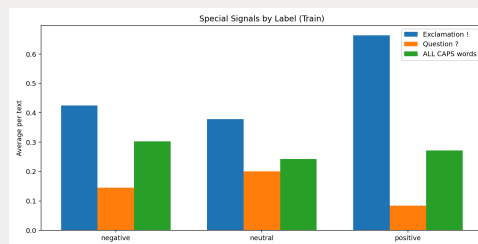
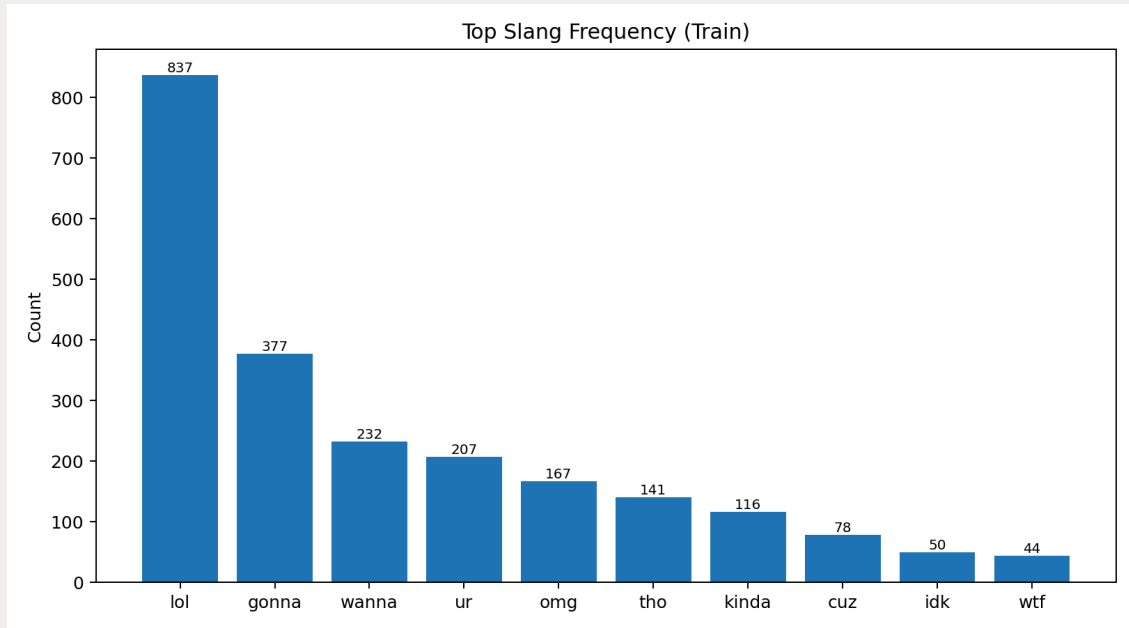
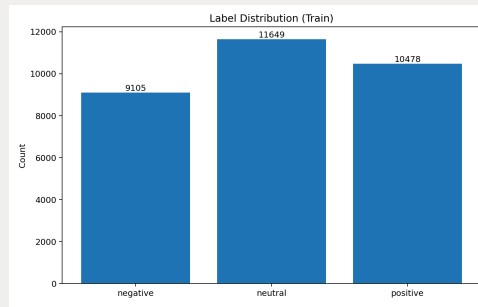
GloVe의 경우는 평균이라는 단순한 풀링 연산이 단어 순서와 부정 표현을 모두 무시한다는 점이 결정적이었다. MiniLM은 sweep으로 찾은 설정으로 다시 학습시켰을 때 Dev가 Epoch 3에서 일찍 고점을 찍고 그 뒤로는 길게 이어지지 못한 것이 컸고, MPNet의 경우는 표현력이 큰 만큼 학습률 민감도가 높아 sweep으로도 안정적인 지점을 잡기 어려웠고, 실제로 다시 학습시켰을 때도 Dev가 50 에폭 내내 큰 진동을 보이면서 Epoch 39에서야 고점(0.6788)을 찍은 것이 그 불안정성을 그대로 보여줬다.

12. 성능 향상을 위해 추가로 시도한 실험들

과제 조건은 이미 충족되었지만 한 번 더 성능을 끌어올려보고 싶어 Exp3 기반 위에 전처리와 핸드크래프트 피처를 더해 v6부터 v10까지 실험을 이어 갔다. 결과는 다음과 같이 정리된다.

- **v6 (전처리 보강):** Test 68.36%

Figure 7. v6 전처리를 위한 data 분석



```

def preprocess_ultimate(text):
    text = text.lower()

    # 1. 백틱 → 어포스트로피 (23.7% 데이터 영향!)
    text = text.replace('`', "'")

    # 2. 검열 별표 → 감성 토큰
    text = text.replace('****', ' bad ')
    text = text.replace('***', ' bad ')

    # 3. 축약어 분리 (백틱 변환 후 정상 작동!)
    text = re.sub(r"won't", "will not", text)
    text = re.sub(r"can't", "cannot", text)
    text = re.sub(r"n't", " not", text)      # don't → do not (핵심!)
    text = re.sub(r"'re", " are", text)
    text = re.sub(r"'ve", " have", text)
    text = re.sub(r"'ll", " will", text)
    text = re.sub(r"'d", " would", text)
    text = re.sub(r"'m", " am", text)

    # 4. 슬랭 정규화 (빈도 기반)
    text = re.sub(r'\bidk\b', 'i do not know', text)
    text = re.sub(r'\bur\b', 'your', text)
    text = re.sub(r'\bnaw\b', 'no', text)
    text = re.sub(r'\bgonna\b', 'going to', text)
    text = re.sub(r'\bwanna\b', 'want to', text)
    text = re.sub(r'\blol\b', 'laughing', text)
    text = re.sub(r'\bomg\b', 'oh my god', text)
    text = re.sub(r'\bwtf\b', 'what the', text)

    return text

# TF-IDF with 완벽한 전처리
vectorizer=TfidfVectorizer(
    max_features=30000,
    preprocessor=preprocess_ultimate, # 🎯 완벽한 전처리!
    min_df=2
)
vectorizer.fit(train_data['text'])
train_v=vectorizer.transform(train_data['text'])
dev_v=vectorizer.transform(dev_data['text'])
test_v=vectorizer.transform(test_data['text'])
input_size=train_v.shape[1]
train_t=torch.FloatTensor(train_v.toarray()).to(device)
dev_t=torch.FloatTensor(dev_v.toarray()).to(device)
test_t=torch.FloatTensor(test_v.toarray()).to(device)
dev_labels_t=torch.tensor(dev_data['label'], dtype=torch.long).to(device)

```



```
print(f'TF-IDF:{train_t.shape}|Features:{input_size}')
```

전처리 효과 확인

```
print(f'\n🔍 전처리 예시:')
test_cases = [
    "That`s really bad!",
    "naw idk what ur talkin about",
    "I don't like this ****",
    "gonna wanna lol"
]
for tc in test_cases:
    print(f' 원본: "{tc}"')
    print(f' 처리: "{preprocess_ultimate(tc)}"')
    print()
```

TF-IDF:torch.Size([31232, 11764])|Features:11764

🔍 전처리 예시:

원본: "That`s really bad!"

처리: "that's really bad!"

원본: "naw idk what ur talkin about"

처리: "no i do not know what your talkin about"

원본: "I don't like this ****"

처리: "i do not like this bad "

원본: "gonna wanna lol"

처리: "going to want to laughing"

- **v7 (n-gram 확장):** Test 68.61%

TF-IDF가 아예 보지 못하는 신호를 직접 더해주기 위해 핸드크래프트 피쳐 6개를 별도로 뽑아 붙였다. 클래스별 평균을 비교해 보면 아래와 같이 클래스별로 구분되는 신호가 분명히 존재한다.

피쳐	negative	neutral	positive
느낌표 수	0.425	0.379	0.664
물음표 수	0.145	0.201	0.084
ALL CAPS 단어 수	0.302	0.242	0.272
텍스트 길이 (단어 수)	장문 neg 많음	-	-
URL 존재 여부	0.023	0.047	0.037
이모티콘 존재 여부	-	-	positive 신호

결과적으로 입력 차원은 **TF-IDF 30000 + 핸드크래프트 6 = 30006**차원으로 늘어났다.

```
def preprocess_text(text):
    text = text.lower()
```

```

text = re.sub(r'http\S+|www\S+', '', text)
text = text.replace('`', '')
text = text.replace('*****', ' bad ')
text = text.replace('***', ' bad ')
text = re.sub(r'!\{3,\}', ' verymuch ! ', text)
text = re.sub(r'(\.)\1\{3,\}', r'\1\1', text)
text = re.sub(r"won't", 'will not', text)
text = re.sub(r"can't", 'cannot', text)
text = re.sub(r"n't", ' not', text)
text = re.sub(r"'re", ' are', text)
text = re.sub(r"'ve", ' have', text)
text = re.sub(r"'ll", ' will', text)
text = re.sub(r"'d", ' would', text)
text = re.sub(r"'m", ' am', text)
slangs = [
    (r'\bidk\b', 'i do not know'), (r'\bur\b', 'your'),
    (r'\bnaw\b', 'no'), (r'\bgonna\b', 'going to'),
    (r'\bwanna\b', 'want to'), (r'\blol\b', 'laughing'),
    (r'\bomg\b', 'oh my god'), (r'\bwtf\b', 'what the'),
    (r'\bugh\b', 'disgusting'), (r'\btho\b', 'though'),
    (r'\bkinda\b', 'kind of'), (r'\bcuz\b', 'because'),
    (r'\bsoo+\b', 'so'), (r'\bthx\b', 'thanks'),
    (r'\byep\b', 'yes'), (r'\byup\b', 'yes'),
    (r'\bnope\b', 'no'), (r'\btbh\b', 'to be honest'),
    (r'\bimo\b', 'in my opinion'),
]
for pat, rep in slangs:
    text = re.sub(pat, rep, text)
return text

def extract_handcraft(texts):
    """TF-IDF가 못 보는 신호 6개"""
    features = []
    for text in texts:
        t = str(text)
        tl = t.lower()
        words = t.split()
        feats = [
            min(t.count('!'), 5),
            # 느낌표 수 (positive↑)
            min(t.count('?'), 5),
            # 물음표 수 (neutral↑)
            sum(1 for w in words if w.isupper() and len(w) > 1),
            # ALL CAPS 단어 수
            min(len(words), 50),
            # 단어 수
            int(bool(re.search(r'http\S+', tl))),
            # URL 포함 여부

```

```

        int(any(e in tl for e in [':)', ':(', ':d', ':/', 'haha', 'heh
e', 'lmao'])), # 이모티콘
    ]
    features.append(feats)
    return np.array(features, dtype=np.float32)

# TF-IDF
vectorizer = TfidfVectorizer(max_features=30000, preprocessor=preprocess_t
ext, min_df=2)
vectorizer.fit(train_data['text'])

# 핸드크래프트 피쳐 확인
hc_train = extract_handcraft(train_data['text'])
print(f'핸드크래프트 피쳐 평균 (neg/neu/pos):')
feat_names = ['느낌표', '물음표', 'ALL_CAPS', '단어수', 'URL', '이모티콘']
labels_arr = np.array(train_labels)
for i, name in enumerate(feat_names):
    neg_m = hc_train[labels_arr==0, i].mean()
    neu_m = hc_train[labels_arr==1, i].mean()
    pos_m = hc_train[labels_arr==2, i].mean()
    print(f' {name}: neg={neg_m:.3f} | neu={neu_m:.3f} | pos={pos_m:.3
f}')

# TF-IDF + 핸드크래프트 concat
def build_features(data_split, fit=False):
    texts = data_split['text']
    tfidf_mat = vectorizer.transform(texts)
    hc_mat = csr_matrix(extract_handcraft(texts))
    combined = hstack([tfidf_mat, hc_mat])
    return torch.FloatTensor(combined.toarray()).to(device)

train_t = build_features(train_data)
dev_t = build_features(dev_data)
test_t = build_features(test_data)
dev_labels_t = torch.tensor(dev_data['label'], dtype=torch.long).to(devic
e)
input_size = train_t.shape[1]
print(f'\n최종 입력 크기: {input_size} (TF-IDF 30000 + 핸드크래프트 6)')

```

```

핸드크래프트 피쳐 평균 (neg/neu/pos):
느낌표: neg=0.385 | neu=0.362 | pos=0.635
물음표: neg=0.142 | neu=0.180 | pos=0.084
ALL_CAPS: neg=0.302 | neu=0.242 | pos=0.272
단어수: neg=17.420 | neu=15.309 | pos=16.994
URL: neg=0.023 | neu=0.047 | pos=0.037
이모티콘: neg=0.043 | neu=0.078 | pos=0.081

```

최종 입력 크기: 11663 (TF-IDF 30000 + 핸드크래프트 6)

- v8 (핸드크래프트 피쳐 + 추가 sweep, 70 에폭) ★: Dev 70.05% | Test 69.13%

v8에서 Sweep을 다시 돌린 이유

v7에서 전처리와 핸드크래프트 피쳐를 추가하면서 입력 공간이 30006차원으로 완전히 바뀌었기 때문에, 기존 best config(lr 5.59e-5, hidden 256)는 raw TF-IDF 기준 값이라 그대로 쓰기 어려웠다. 또한 v7 학습 곡선을 보면 Epoch 36에서 Dev best(0.6907)을 찍은 뒤 하락했는데, 학습률이 너무 낮아 수렴이 느리고 은닉층 크기를 더 키울 여지도 보였다. 그래서 새 입력에 맞는 최적 config를 W&B Sweep으로 다시 탐색해야 했다.

v8 전략 요약

v7의 전처리·핸드크래프트 기반은 그대로 유지한 채 W&B Sweep(Bayesian 탐색)을 붙여 하이퍼파라미터를 자동 탐색했다. count=15로 여러 run을 돌린 뒤 best config를 뽑아 재학습하는 “최종 탐색용” 버전이다.

```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = text.replace('`', '')
    text = text.replace('*****', ' bad ')
    text = text.replace('***', ' bad ')
    text = re.sub(r'!{3,}', ' verymuch ! ', text)
    text = re.sub(r'(\.)\1{3,}', r'\1\1', text)
    text = re.sub(r"won't", 'will not', text)
    text = re.sub(r"can't", 'cannot', text)
    text = re.sub(r"n't", ' not', text)
    text = re.sub(r"'re", ' are', text)
    text = re.sub(r"'ve", ' have', text)
    text = re.sub(r"'ll", ' will', text)
    text = re.sub(r"'d", ' would', text)
    text = re.sub(r"'m", ' am', text)
    slangs = [
        (r'\bidk\b', 'i do not know'), (r'\bur\b', 'your'),
        (r'\bnaw\b', 'no'), (r'\bgonna\b', 'going to'),
        (r'\bwanna\b', 'want to'), (r'\blo\b', 'laughing'),
        (r'\bomg\b', 'oh my god'), (r'\bwtf\b', 'what the'),
        (r'\bugh\b', 'disgusting'), (r'\btho\b', 'though'),
        (r'\bkinda\b', 'kind of'), (r'\bcuz\b', 'because'),
        (r'\bsoo+\b', 'so'), (r'\bthx\b', 'thanks'),
        (r'\byep\b', 'yes'), (r'\byup\b', 'yes'),
        (r'\bnope\b', 'no'), (r'\btbh\b', 'to be honest'),
        (r'\bimo\b', 'in my opinion'),
    ]
    for pat, rep in slangs:
        text = re.sub(pat, rep, text)
    return text
```

```

def extract_handcraft(texts):
    features=[]
    for text in texts:
        t=str(text)
        tl=t.lower()
        words=t.split()
        features.append([
            min(t.count('!'),5),
            min(t.count('?'),5),
            sum(1 for w in words if w.isupper() and len(w)>1),
            min(len(words),50),
            int(bool(re.search(r'http\S+',tl))),
            int(any(e in tl for e in [':','(',':d',':/','haha','hehe','l
mao'])))
        ])
    return np.array(features,dtype=np.float32)

# 전처리 + 핸드크래프트 피쳐 빌드 (Sweep에서 재사용)
vectorizer=TfidfVectorizer(max_features=30000,preprocessor=preprocess_text,min_df=2)
vectorizer.fit(train_data['text'])

def build_features(data_split):
    tfidf_mat=vectorizer.transform(data_split['text'])
    hc_mat=csr_matrix(extract_handcraft(data_split['text']))
    return torch.FloatTensor(hstack([tfidf_mat,hc_mat]).toarray()).to(device)

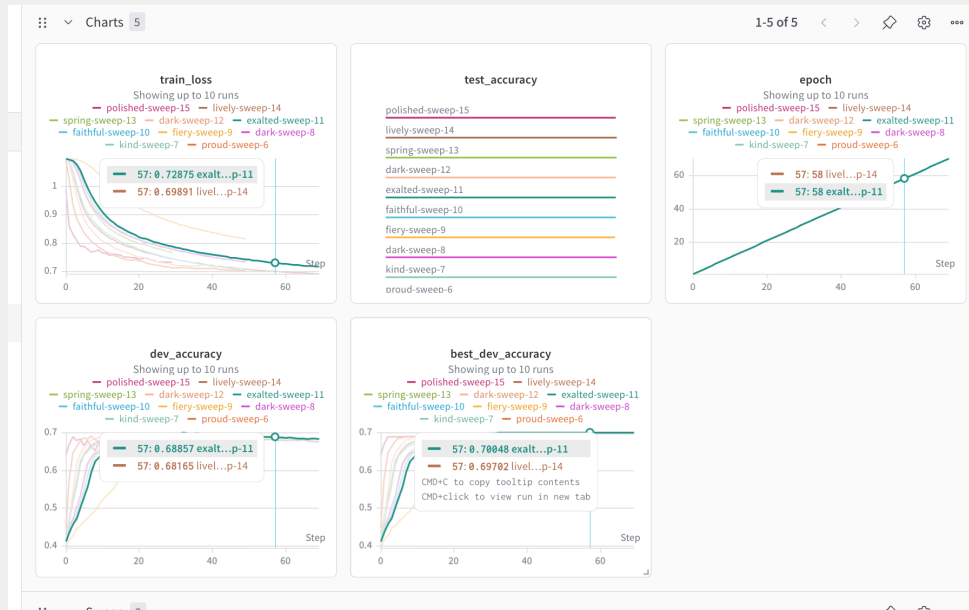
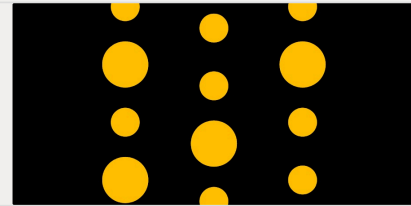
train_t=build_features(train_data)
dev_t=build_features(dev_data)
test_t=build_features(test_data)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=train_t.shape[1]
print(f'입력 크기:{input_size}')

```

imeanseo_

Weights & Biases, developer tools for machine learning

https://wandb.ai/imeanseo_/nlp-hw1/sweeps/8oy7hj3j?nw=nwuseraileen02ko



Best config: {'dropout': 0.4, 'batch_size': 256, 'num_epochs': 70, 'hidden_size': 1000, 'weight_decay': 0.0001, 'learning_rate': 3.649238347809796e-05}

🚩 최종 학습...

🌟 Epoch 1 | Dev: 0.4110 | NEW BEST!
🌟 Epoch 2 | Dev: 0.4359 | NEW BEST!
🌟 Epoch 3 | Dev: 0.4574 | NEW BEST!
🌟 Epoch 4 | Dev: 0.4726 | NEW BEST!
🌟 Epoch 5 | Dev: 0.5036 | NEW BEST!
🌟 Epoch 6 | Dev: 0.5285 | NEW BEST!
🌟 Epoch 7 | Dev: 0.5562 | NEW BEST!
🌟 Epoch 8 | Dev: 0.5967 | NEW BEST!
🌟 Epoch 9 | Dev: 0.6250 | NEW BEST!
🌟 Epoch 10 | Dev: 0.6340 | NEW BEST!
🌟 Epoch 11 | Dev: 0.6461 | NEW BEST!
🌟 Epoch 12 | Dev: 0.6565 | NEW BEST!
🌟 Epoch 13 | Dev: 0.6567 | NEW BEST!
🌟 Epoch 14 | Dev: 0.6630 | NEW BEST!
🌟 Epoch 15 | Dev: 0.6701 | NEW BEST!
🌟 Epoch 16 | Dev: 0.6707 | NEW BEST!
🌟 Epoch 17 | Dev: 0.6728 | NEW BEST!

✨ Epoch 18 | Dev: 0.6772 | NEW BEST!
 ✨ Epoch 19 | Dev: 0.6805 | NEW BEST!
 ✨ Epoch 20 | Dev: 0.6824 | NEW BEST!
 ✨ Epoch 21 | Dev: 0.6870 | NEW BEST!
 ✨ Epoch 23 | Dev: 0.6901 | NEW BEST!
 ✨ Epoch 24 | Dev: 0.6934 | NEW BEST!
 ✨ Epoch 26 | Dev: 0.6941 | NEW BEST!
 ✨ Epoch 27 | Dev: 0.6953 | NEW BEST!
 ✨ Epoch 28 | Dev: 0.6955 | NEW BEST!
 ✨ Epoch 29 | Dev: 0.6964 | NEW BEST!
 ✨ Epoch 31 | Dev: 0.6974 | NEW BEST!
 ✨ Epoch 33 | Dev: 0.7005 | NEW BEST!

✅ 저장: best_model_exp3_v8_final.pt
 📊 Dev: 0.7005 | Test: 69.13%

- v9 (피처 조합 조정 + 스케일 정규화): Test 68.76%

v8에서 발견한 문제

v8은 Dev 70.05%까지 올라갔지만 Test 69.13%로 약 0.92%p의 격차가 남았다. 원인을 살펴보니 핸드크래프트 피처와 TF-IDF의 스케일 차이가 의심됐다. TF-IDF는 0.0~1.0 범위였는데 단어 수 같은 핸드크래프트 피처는 0~50 범위로 약 50배 더 컸고, MLP가 이 큰 스케일의 피처에 과도하게 의존하면서 일반화가 떨어지는 것으로 보였다.

v9 개선 방향

StandardScaler로 핸드크래프트 피처를 정규화해 평균 0·표준편차 1로 맞추면, TF-IDF와 동일한 스케일에서 학습시켜 dev-test 격차를 줄이는 것이 목표였다.

```

# 핸드크래프트 피처 추출 후 StandardScaler 적용 (train 기준으로 fit)
hc_train_raw=extract_handcraft(train_data['text'])
hc_dev_raw=extract_handcraft(dev_data['text'])
hc_test_raw=extract_handcraft(test_data['text'])

scaler=StandardScaler()
hc_train_scaled=scaler.fit_transform(hc_train_raw) # train으로만 fit!
hc_dev_scaled=scaler.transform(hc_dev_raw)
hc_test_scaled=scaler.transform(hc_test_raw)

# 스케일 확인
feat_names=['느낌표', '물음표', 'ALL_CAPS', '단어수', 'URL', '이모티콘']
print('핸드크래프트 피처 정규화 전/후 범위:')
for i,name in enumerate(feat_names):
    before=f'{hc_train_raw[:,i].min():.1f}~{hc_train_raw[:,i].max():.1f}'
    after=f'{hc_train_scaled[:,i].min():.2f}~{hc_train_scaled[:,i].max():.2f}'
    print(f' {name}: {before} → {after}')
  
```

```
def build_features(tfidf_mat, hc_scaled):
    combined=hstack([tfidf_mat, csr_matrix(hc_scaled)])
    return torch.FloatTensor(combined.toarray()).to(device)

train_t=build_features(vectorizer.transform(train_data['text']),hc_train_scaled)
dev_t=build_features(vectorizer.transform(dev_data['text']),hc_dev_scaled)
test_t=build_features(vectorizer.transform(test_data['text']),hc_test_scaled)
dev_labels_t=torch.tensor(dev_data['label'],dtype=torch.long).to(device)
input_size=train_t.shape[1]
print(f'\n입력 크기: {input_size} (TF-IDF 30000 + 핸드크래프트 6, 정규화됨)')
```

핸드크래프트 피쳐 정규화 전/후 범위:

느낌표: 0.0~59.0 → -0.40~47.66
 물음표: 0.0~98.0 → -0.17~115.83
 ALL_CAPS: 0.0~60.0 → -0.23~51.73
 단어수: 1.0~391.0 → -0.95~20.90
 URL: 0.0~1.0 → -0.20~5.12
 이모티콘: 0.0~1.0 → -0.27~3.68

입력 크기: 11663 (TF-IDF 30000 + 핸드크래프트 6, 정규화됨)

- v10 (v8 best config + epochs 70 → 100, 마지막 시도): Dev 70.05% | Test 69.13%

H, LR, D, WD, EP, BS = 1000, 3.649238347809796e-05, 0.4, 1e-4, 100, 256

v8에서는 기존 Exp3 sweep 결과와는 다른 설정(은닉층 크기 1000, 학습률 3.65e-5, 드롭아웃 0.4, 배치 크기 256, weight decay 1e-4, 70 에폭)이 최적으로 선택되어 Dev 0.7005까지 올라왔지만, Test는 69.13%에 멈춰 dev와 test 사이에 약 0.92%p의 격차가 남아 있었다. v10은 이 v8 best config를 그대로 둔 채 epoch만 70에서 100으로 늘려, 학습률이 매우 낮은 만큼 더 천천히 수렴시켜 dev의 상승분이 test에도 반영되도록 해본 마지막 시도였다(v9에서 스케일 정규화가 오히려 역효과였기 때문에 이번엔 정규화는 다시 뺐다). 결과는 Dev 70.05% / Test 69.13%로 v8과 완전히 같아서, 이 설정이 만들 수 있는 상한선이 69.13% 근처이며 dev-test 격차는 단순한 epoch 연장으로 메워지지 않는다는 점을 확인하고 실험을 마무리했다.

13. 최종 모델

최종 제출 모델은 **Exp3-v8**이다. Exp3에서 찾아낸 TF-IDF + W&B Sweep 설정을 출발점으로 잡은 뒤 전처리 보강과 n-gram 확장, 핸드크래프트 피쳐를 구조에 더하고, v8 단계에서 sweep을 한 번 더 돌려 얻은 설정(은닉층 크기 1000, 학습률 3.65e-5, 드롭아웃 0.4, 배치 크기 256, weight decay 1e-4, 70 에폭)을 그대로 사용한 버전이다. test 정확도는 **69.13%**로 베이스라인 대비 **+1.75%p**의 향상을 안정적으로 재현할 수 있으며, 다섯 개의 본 실험과 추가 개선 실험을 통틀어 가장 높은 성능이다. v10에서는 같은 best config를 그대로 둔 채 epoch만 70에서 100으로 늘려 dev의 0.7005가 test에도 더 반영되도록 천천히 수렴시켰지만, 결과는 Dev 70.05% / Test 69.13%로 v8과 완전히 동일해 이 설정이 이 구조에서 도달 가능한 상한선임을 다시 한 번 확인했다.

과제에서 명시적으로 요구한 "MLP 구조 자체는 변경하지 않았다"는 조건도 모든 실험에 걸쳐 일관되게 지켰다. 입력 표현, 학습 하이퍼파라미터, 그리고 일부 전처리를 변경했다.

14. 이번 과제를 통한 인사이트

이번 과제를 통해 가장 크게 느낀 점은, 모델이나 임베딩이 더 정교해진다고 해서 성능이 그대로 따라 올라주지는 않는다는 사실이다. 768차원의 MPNet, 384차원의 MiniLM, 300차원의 GloVe 같이 이론적으로는 더 풍부한 표현을 가진 모델들이 오히려 BoW나 TF-IDF보다 낮게 나온 것이 대표적인 예다. 우리 데이터의 신호가 결국 "감성 키워드가 있는가, 있다면 얼마나 중요한 키워드인가"에 가깝고, MLP 구조를 그대로 둔 채 풍부한 임베딩을 단순 평균이나 단일 은닉층으로 압축하는 이 세팅에서는 그 풍부함이 제대로 살아남지 못했다는 점을 직접 확인한 셈이다.

두 번째로, 성능을 올리는 레버를 어느 순서로 잡아야 효율적인지에 대한 감각을 얻었다. 같은 BoW를 둔 채 sweep으로 학습 하이퍼파라미터만 자동 탐색한 것만으로 +0.88%p의 향상이 나왔다는 점은, 표현을 건드리기 전에 학습 설정만 잘 다듬어도 사실상 가장 큰 즉각 향상이 가능하다는 의미였다. 반면 그 위에서 표현을 BoW에서 TF-IDF로 바꾸는 한 걸음은 +0.04%p의 작은 추가 향상에 그쳤다. 그럼에도 TF-IDF를 쓴 것이 의미가 있었던 이유는, 그 뒤 v6부터 v10까지 더한 전처리·n-gram 확장·핸드크래프트 피처가 TF-IDF 위에서야 비로소 의미 있게 누적되며 최종 +1.75%p까지 끌어올렸기 때문이다. 실제로 같은 전처리·핸드크래프트 피처 아이디어를 BoW 위에 그대로 얹어본 추가 ablation(Exp2-v2)에서는 Test 68.70%로 +0.44%p 향상에 그친 반면, 같은 개선을 TF-IDF 위에 얹은 Exp3-v8은 +0.83%p를 더 얻어 결과적으로 거의 두 배의 효과를 냈다. 즉, sweep이 즉각적인 큰 레버였다면 TF-IDF는 그 자체로 큰 향상을 만들기보다는 그 위에서 같은 개선 아이디어가 두 배로 살아나도록 받쳐주는 후속 세부 개선의 발판으로서 가치 있는 선택이었다.

세 번째로, dev와 test 사이의 격차를 줄이는 것이 생각보다 훨씬 어렵다는 점을 배웠다. v8에서 Dev가 0.7005까지 올라갔지만 Test는 69.13%에서 멈춰 있었고, v10에서 epoch을 70에서 100으로 늘려도 Dev/Test 모두 동일한 수치에 머무르며 더 이상 올라가지 않았다. 이는 dev에서의 성능 향상이 그대로 test 일반화로 이어지지 않을 수 있으며, 이 계열의 구조와 데이터로는 69%대 중반이 현실적인 상한선일 수 있다는 감각을 주었다. 결국 결정적인 주어진 조건의 변화가 없으면, 단순한 파라미터 조정만으로는 일반화 격차를 메우기 어렵다는 점을 실감했다.

마지막으로, 모든 실험에서 시드를 고정해 두었기 때문에 베이스라인을 언제 다시 돌려도 같은 67.38%가 나오는 것을 확인할 수 있었고, v8과 v10이 Dev/Test 모두 소수점 아래까지 일치했다는 점도 이 설정 덕분이었다. 시드 고정만은 단순한 관행 같아 보이지만, 한 번의 우연한 고점과 계속 재현 가능한 결과를 구분해주는 핵심 장치임을 이번 과제를 통해 철저히 체감했다.